PSX Stock Insight

Explanation guide and viva Q&A, written for someone with no ML background

1. The project in one minute

You have years of daily price history for 10 companies on the Pakistan Stock Exchange (PSX). You turn that history into numbers a model can learn from, train two algorithms that you built yourself with NumPy (Random Forest Regressor and KNN), then test them on recent data they have never seen and report honestly how accurate they are.

> **Important:** this is not a tool that says "buy this stock". It measures how well machine learning can predict something noisy and real. Saying this clearly at the start protects you from the most dangerous question.

The whole pipeline

Step	What happens	Output
1. Collect	Read the CSV files of 10 stocks from the Kaggle dataset; write a short JSON profile per company.	Raw data
2. Clean	Fix dates, remove missing and duplicate rows, log problems.	Clean data
3. Features	Calculate indicators (return, SMA, RSI, volatility, volume change).	Table of numbers
4. Targets	Define what to predict: next-day return and next 5-day volatility.	Labels
5. Split	Older data for training, newest data for testing.	Train / test sets
6. Train	Random Forest Regressor (trend and risk) and KNN (similar stocks), from scratch.	Trained models
7. Test	Predict on test data and compare with reality.	Accuracy and error numbers
8. Save	Store models with pickle so they can be reloaded.	.pkl files

2. The data

Dataset: "Pakistan Stock Exchange Complete" on Kaggle (by mirfanazam). It has daily data for 314 PSX stocks from Jan 2018 to May 2022. Link: https://www.kaggle.com/datasets/mirfanazam/pakistan-stock-exchange-complete

Each CSV file is **one company**, and the file name is its stock symbol (for example FFC.csv is Fauji Fertilizer). Each row is one trading day with Date, Open, High, Low, Close and Volume. You only use 10 files:

- **Fertilizer and chemicals:** ENGRO, FFC, EFERT
- **Cement:** DGKC, FCCL
- **Banking:** HBL, BAFL, BAHL
- **Energy:** HUBC, ATRL

The JSON files hold a short written profile per company (sector, description, business drivers). They are small and written by you. Other data sources mentioned in the proposal (PSX Data Portal at https://dps.psx.com.pk, and yfinance) can be used for newer data.

3. Features: turning prices into numbers a model can learn

Raw prices are a poor input. A price of 500 means something very different for two different companies, and prices keep drifting upwards or downwards. So we convert prices into **indicators** that describe behaviour instead.

Feature	Plain meaning	How it is calculated
Daily return	How much the price changed today, in percent.	(today close - yesterday close) / yesterday close
SMA 20 / SMA 50	The average closing price of the last 20 or 50 days. Shows the trend and smooths noise.	Mean of the last 20 (or 50) closes
RSI (14 days)	A momentum score from 0 to 100. Above 70 usually means the stock rose a lot recently; below 30 means it fell a lot.	RSI = 100 - 100 / (1 + RS), where RS = average gain / average loss over 14 days
Volatility	How wildly the returns have been swinging. This is the measure of risk.	Standard deviation of the last 20 daily returns
Volume change	Is trading activity rising or falling?	(today volume - yesterday volume) / yesterday volume

Targets: what the models try to predict

- **Trend:** tomorrow's return. If the predicted value is above 0, the trend is "up"; otherwise "down".
- **Risk:** the volatility over the next 5 trading days. It is then grouped into Low, Medium or High using cut-offs (for example thirds of the training data).

We predict volatility for risk because volatile periods tend to continue for a while, so it is far more predictable than the direction of price.

Train/test split by time

Train on the older part (for example 2018 to 2021) and test on the newest part (2021 to 2022). This copies real life: you can only learn from the past. A random split would let the model train on days that come *after* the days it is tested on. That leaks the future and produces fake accuracy.

4. The models, explained simply

We use two algorithms. Both are written from scratch with NumPy, which means you code the logic yourself instead of calling a library.

4.1 Decision tree (the building block)

A decision tree is a flowchart of yes/no questions about the features. Think of the game "20 questions". At the end of the flowchart (a "leaf") it gives a number. For a regression tree, that number is the **average target value of the training days that ended up in that leaf**.

```
Is volatility > 2% ?
   YES: Is RSI > 65 ?
          YES: predict next-day return = -0.4%
          NO : predict next-day return = +0.1%
   NO : predict next-day return = +0.2%
```

How does it pick the questions? At each step the tree tries many (feature, threshold) pairs, such as "volatility > 2%" or "RSI > 65". For each pair it splits the data into two groups. It keeps the pair that makes the two groups most *similar inside* (the target values in each group are close together, meaning the lowest variance). It then repeats on each group until a stopping rule is reached, such as a maximum depth or a minimum number of rows per leaf.

Weakness: one deep tree memorizes the training data (overfitting), so it does badly on new data. This is why we use a forest.

4.2 Random Forest Regressor

A Random Forest is **many decision trees whose answers are averaged**. It is the "ask 100 different friends and take the average" idea. One friend may be wrong, but their random mistakes cancel out in the average.

> **Two kinds of randomness make each tree different:**
> 1) **Bootstrap sampling:** each tree is trained on a random sample of the training rows, drawn *with replacement* (some rows appear twice, some not at all).
> 2) **Random feature subsets:** at every split the tree is only allowed to look at a random handful of the features.
> Different trees therefore make different mistakes, and averaging them gives a steadier answer.

Steps to build it (what your code does):

- Repeat N times (for example 50 or 100): draw a bootstrap sample of the training rows and grow one tree on it, using random feature subsets at each split.
- To predict for a new day: send its features down every tree, collect all the answers, and **take the average**.

In this project there are two forests: one predicts **next-day return** (the sign gives the trend) and one predicts **volatility over the next 5 days** (grouped into Low / Medium / High risk).

Why it suits this data: it handles non-linear patterns (for example "RSI matters only when volatility is high"), needs little tuning, does not need features scaled, and the averaging reduces overfitting.

4.3 K-Nearest Neighbors (KNN)

KNN has no training in the usual sense. It simply **remembers all the data** and answers a question by looking at the **k closest examples**. The idea is "tell me who your neighbours are and I will tell you who you are".

Where we use it: to answer "which stocks are similar to this one?". Each stock is described by a few numbers, such as its average volatility, average return and average volume. Stocks whose numbers are close are neighbours.

The steps:

- **Describe each stock with numbers** (average volatility, average return, average volume).
- **Scale the numbers** to the same range (for example 0 to 1). This is essential: volume is in millions while volatility is a small percentage, so without scaling, volume would completely dominate the distance.
- **Measure the distance** from the chosen stock to every other stock using Euclidean distance (straight-line distance): square root of the sum of squared differences.
- **Sort by distance** and take the k smallest. Those are the most similar stocks.

Worked example (values already scaled to 0-1; the question is "which stocks are similar to BAFL?", with k = 2):

Stock	Volatility	Volume	Distance to BAFL (0.35, 0.65)
HBL	0.30	0.70	$\text{sqrt}(0.05^2 + 0.05^2) = 0.071$
BAHL	0.32	0.60	$\text{sqrt}(0.03^2 + 0.05^2) = 0.058$
HUBC	0.55	0.40	$\text{sqrt}(0.20^2 + 0.25^2) = 0.320$
DGKC	0.75	0.20	$\text{sqrt}(0.40^2 + 0.45^2) = 0.602$

The two smallest distances are BAHL (0.058) and HBL (0.071), so the 2 nearest neighbours of BAFL are BAHL and HBL. Both are banks, which is exactly the sanity check we want. DGKC (cement) is far away.

Choosing k: a small k (1) is sensitive to noise; a large k blurs the differences. Try a few values (3, 5, 7) and pick the one whose results make the most sense.

KNN as a predictor: the same idea can predict a number: find the k most similar past days and average their next-day returns. You can mention this as an extension, but the proposal uses KNN mainly for similarity.

4.4 Random Forest vs KNN at a glance

	Random Forest Regressor	KNN
Job in this project	Predict next-day return (trend) and 5-day volatility (risk)	Find similar stocks
How it learns	Builds many trees from the training data	Just stores the data
How it answers	Averages the answers of all trees	Looks at the k closest examples
Needs scaling?	No	Yes, always
Main risk	Overfitting if trees are too deep	Wrong results if features are not scaled or k is poor

5. How we judge the models (evaluation)

Metric	Meaning	Tiny example
Direction accuracy	Share of days where the predicted sign (up/down) was right.	8 correct out of 15 days = 53%
Baseline	A dumb rule to beat, such as "always predict up". If 52% of test days were up, this baseline scores 52%.	Your model must beat it to be useful
MAE	Average size of the errors.	Errors 0.5, 1, 1 give MAE = 2.5 / 3 = 0.83

Metric	Meaning	Tiny example
RMSE	Like MAE but squares errors first, so big mistakes count more.	sqrt((0.25 + 1 + 1) / 3) = 0.87
Risk level match	How often predicted Low/Medium/High equals the actual one.	Reported as a percentage
KNN check	Manual check that results make sense (banks with banks, cement with cement).	See the worked example above

What results to expect: direction accuracy is likely around 50 to 56%, which is normal for stocks. Volatility prediction should look noticeably better. Say this yourself before anyone asks; honest reporting earns more respect than a suspiciously high number.

Saving models: trained models are saved with pickle (.pkl files) so they can be loaded again without retraining.

6. Words you must know

Term	One-line meaning
Overfitting	The model memorizes training data and fails on new data.
Data leakage	The model gets information it would not have in real life (for example future data).
Bootstrap	Sampling rows with replacement to create a different training set for each tree.
Feature	An input number, such as RSI or volatility.
Target	The thing we want to predict, such as tomorrow's return.
Regression	Predicting a number (as opposed to picking a category).
Baseline	A simple rule your model must beat to prove it is useful.
Scaling	Putting features on the same range so none dominates.

7. Questions your instructor may ask (with answers)

Q1. Why not predict the actual price?

Prices trend and drift, so a model can look accurate just by copying yesterday's price. Returns and volatility are the meaningful targets, and they are easier to compare fairly across stocks.

Q2. Is about 52% direction accuracy bad?

No. Short-term stock movement is very noisy, so a result slightly above chance is normal. What matters is that we compare with a baseline and report honestly.

Q3. Why split by time instead of randomly?

A random split would train on days that come after test days, leaking the future and giving fake accuracy. A time-based split copies real use.

Q4. What is data leakage and how do you avoid it?

Using information the model would not have at prediction time. Our features use only data up to today, and the target is strictly tomorrow or later. We also split by time.

Q5. Why Random Forest?

It handles non-linear relationships, needs little tuning, does not need scaled features, and averaging many trees reduces overfitting compared with a single tree.

Q6. What is overfitting and how do you reduce it?

Memorizing the training data so the model fails on new data. The forest reduces it with bootstrapping, random feature subsets and averaging. Limiting tree depth also helps, and the time-based test set shows whether it happened.

Q7. How does a decision tree choose where to split?

It tries many feature/threshold pairs and keeps the one that makes the two resulting groups most similar inside (lowest variance of the target).

Q8. Why does averaging many trees help?

Each tree makes different random mistakes. When their answers are averaged, the mistakes tend to cancel, so the result is more stable than any single tree.

Q9. How does KNN work and how do you choose k?

It stores the data and, for a query, finds the k closest items by Euclidean distance. We try several values of k (such as 3, 5, 7) and choose the one with the most sensible results.

Q10. Why must features be scaled for KNN?

KNN is based on distance. A feature with large numbers, such as volume, would dominate the distance and hide the others, so all features are scaled to the same range.

Q11. Why build the models from scratch instead of using scikit-learn?

To show we understand how the algorithms work: splitting, bootstrapping, distance and averaging. If asked, we can compare our results with a library version as a sanity check.

Q12. How do you define Low, Medium and High risk?

By thresholds on the predicted volatility, such as thirds of the training data. It is a simple rule applied on top of a numeric prediction, and we state that clearly.

Q13. What does RSI mean?

A momentum score from 0 to 100 based on average gains versus average losses over 14 days. Above 70 usually means the stock rose a lot recently; below 30 means it fell a lot.

Q14. MAE vs RMSE?

MAE is the average size of the errors. RMSE squares the errors first, so large mistakes are punished more. Reporting both shows whether a few big errors are hiding in the average.

Q15. How do you know the model is better than guessing?

By comparing with baselines: "always up" for direction, and "tomorrow's volatility equals today's" for risk. The model has to beat these.

Q16. Why does the data stop in 2022?

The Kaggle dataset covers Jan 2018 to May 2022. Newer data can be added from the PSX Data Portal if needed.

Q17. Why only 10 stocks?

To keep the project manageable and the company profiles accurate. The 10 stocks cover four sectors, which is enough to test the models and the similarity search.

Q18. Can this be used for real trading?

No. It is a learning and evaluation project, and the proposal states it is not financial advice.

Q19. What are the limitations?

Noisy data, modest direction accuracy, no news or sentiment, only 10 stocks, and company profiles written by hand.

Q20. What if your accuracy comes out near 50%?

That is an honest and expected result for stock direction. We report it, explain why, and point out that volatility (risk) is predicted better.

8. Suggested way to present (about 3 minutes)

- **Problem:** PSX is volatile and beginners cannot judge risk easily.
- **Data:** Kaggle dataset, 10 stocks, 4 sectors, daily prices.
- **Features and targets:** RSI, SMA, returns, volatility; predict next-day return and 5-day volatility.
- **Models:** Random Forest Regressor and KNN, built from scratch with NumPy.
- **Evaluation:** time-based split, accuracy versus baseline, MAE and RMSE.
- **Honest expectation:** trend accuracy will be modest, volatility prediction better, and this is not financial advice.